

Flowboard

A polished Kanban task board with guest authentication, drag-and-drop status updates, and collaboration features — built for the Next Play Games internship challenge.

[React + TypeScript](#)[Supabase Auth + RLS](#)[Vite + Tailwind](#)[Deployed on Vercel](#)

Candidate: Sailesh Pulukuri · **Document:** sailesh_pulukuri_task_manager_assessment.pdf

LIVE DEMO

<https://kanban-board-pi-drab.vercel.app/>

Hosted on Vercel. Opens with an anonymous guest session and persists tasks in Supabase.

GITHUB REPOSITORY

<https://github.com/saileshpulukuri/kanban-board>

Public source code, schema SQL, README, and setup instructions.

Submission checklist

- Kanban board with To Do / In Progress / In Review / Done
- Drag-and-drop status updates
- Create / edit / delete tasks
- Guest accounts (Supabase anonymous auth)
- Row Level Security enabled
- Live hosted frontend (Vercel)
- GitHub repository shared
- Full database schema included
- Advanced features (team, labels, comments, activity, filters, stats)
- Design-focused UI inspired by Linear / Asana

Project Overview

Flowboard is a full-stack Kanban task manager where users can create work items, move them across board columns, assign teammates, attach labels, leave comments, and track activity — all behind a guest session so each visitor only sees their own data.

Problem & goal

Build a board that feels production-ready — not a generic todo list. Design quality was treated as a first-class requirement: clear hierarchy, intentional typography/color, smooth drag interactions, and thoughtful empty / loading / error states.

Core user flow

1. App launches and creates an anonymous guest session
2. User creates tasks with priority, due date, assignee, labels
3. Tasks are dragged between columns to update status
4. Detail drawer supports edits, comments, and activity history

Design decisions

- **Brand-first UI:** “Flowboard” is the primary visual signal in the header (Instrument Serif + Sora).
- **Palette:** cool slate neutrals with a teal accent — calm and professional rather than a generic purple theme.
- **Hierarchy:** sidebar for team/labels/stats; main canvas reserved for the four-column board.
- **Cards:** priority pills, due-soon / overdue indicators, and assignee avatars without clutter.
- **Motion:** staggered column entrance, elevated drag overlay, and a sliding task detail drawer.

Required functionality delivered

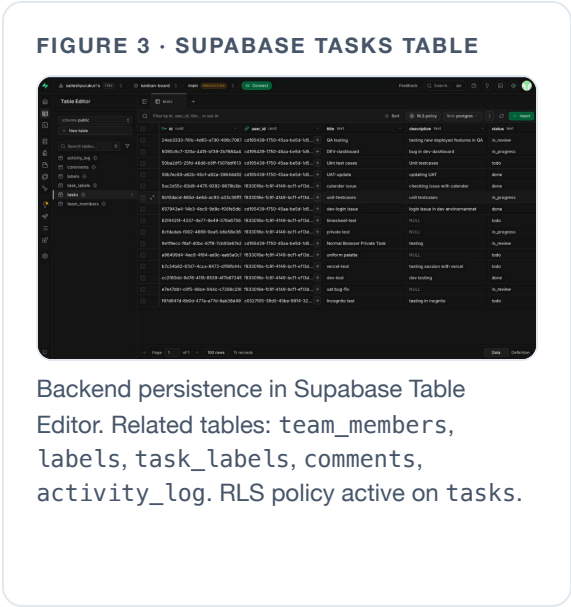
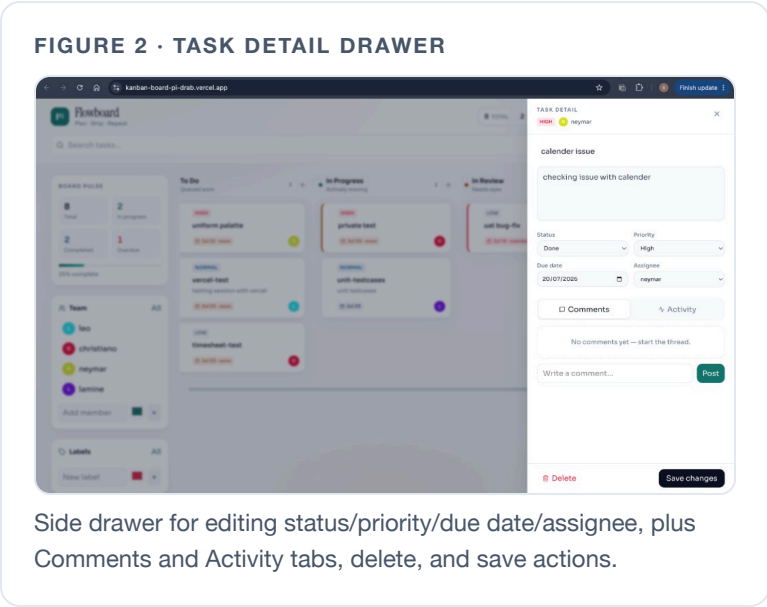
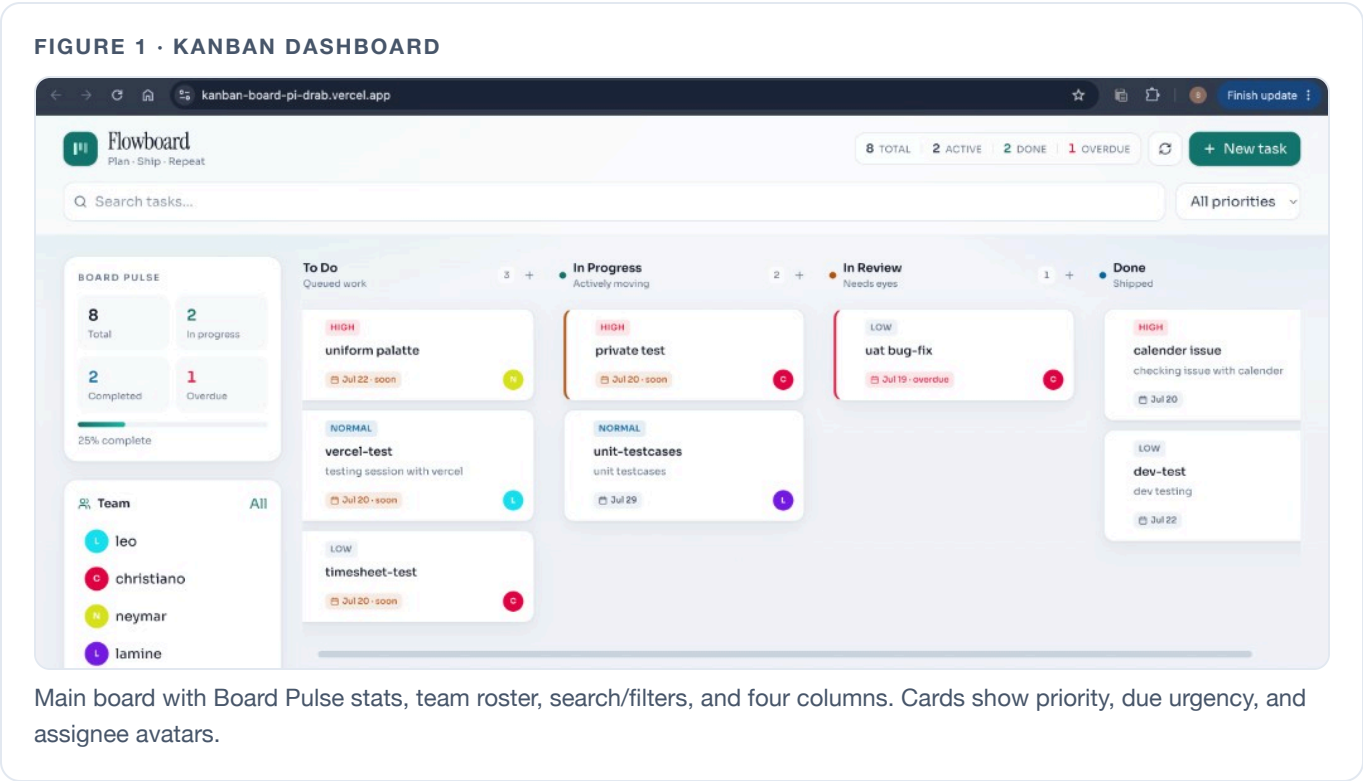
REQUIREMENT	IMPLEMENTATION
Kanban columns	To Do, In Progress, In Review, Done with empty states and counts
Drag & drop	@dnd-kit; status updates optimistically, then persisted to Supabase
Task create/edit	Modal + detail drawer for title, description, priority, due date, assignee, labels
Guest accounts	Supabase anonymous sign-in on first launch; user_id on every owned row
Persistence + security	Postgres tables with RLS policies; only public anon/publishable key in frontend

Advanced features included

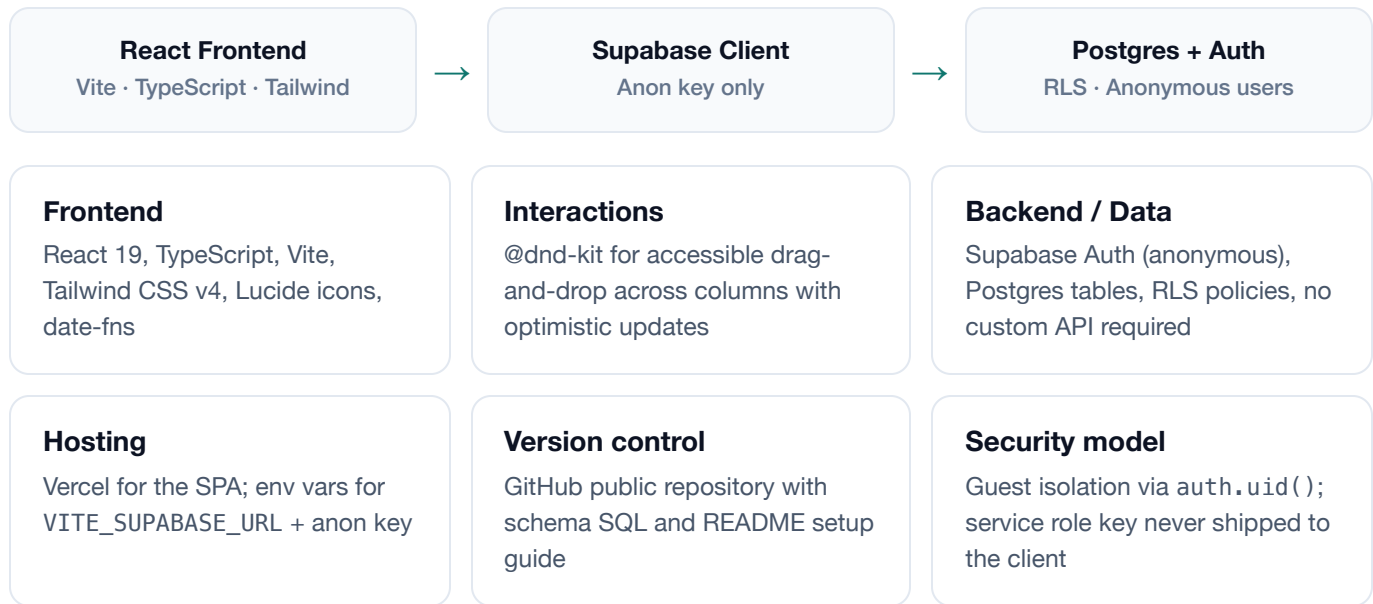
- Team members & assignees (avatars on cards)
 - Task comments (separate comments table)
 - Activity log timeline in task detail
 - Custom labels + multi-label assignment
- Due date urgency badges (soon / overdue)
 - Search + priority / assignee / label filters
 - Board pulse stats + completion progress
 - Loading skeletons and error/empty states

Product Screenshots

Live production UI on Vercel and the corresponding Supabase tasks table in the dashboard.



Architecture & Technologies



Why this architecture

- **Speed + free tier:** Calling Supabase directly from the frontend keeps the stack simple and \$0.
- **Security by default:** RLS enforces ownership even if the client is modified.
- **Clear ownership:** every task/member/label/comment/activity row is tied to the guest `user_id`.
- **Deploy simplicity:** static Vite build on Vercel with two public environment variables.

Key frontend modules

AREA	RESPONSIBILITY
src/components/Board.tsx	Column layout + drag/drop orchestration
src/hooks/useBoard.tsx	Shared board state, filters, stats, CRUD actions
src/lib/api.ts	Supabase reads/writes for tasks, members, labels, comments, activity
supabase/schema.sql	Tables, indexes, triggers, and RLS policies

Guest behavior: User A and User B each receive separate anonymous sessions. Because RLS filters by `auth.uid()`, their boards never leak into each other.

Database Schema & Security

Schema lives in `supabase/schema.sql`. Anonymous sign-in must be enabled under Authentication → Providers → Anonymous.

TABLE	PURPOSE / KEY FIELDS
tasks	Core work items: <code>id</code> , <code>user_id</code> , <code>title</code> , <code>description</code> , <code>status</code> , <code>priority</code> , <code>due_date</code> , <code>assignee_id</code> , <code>position</code> , <code>created_at</code> , <code>updated_at</code> . Status values: <code>todo</code> <code>in_progress</code> <code>in_review</code> <code>done</code> .
team_members	Per-guest roster: <code>name</code> , <code>color</code> , <code>avatar_url</code> for assignees.
labels / task_labels	Custom tags and many-to-many assignment to tasks.
comments	Chronological discussion thread per task.
activity_log	Timeline events (created, status moved, assigned, commented, updated).

Row Level Security

- RLS enabled on all application tables
- Policies allow access only when `auth.uid() = user_id`
- `task_labels` is authorized through ownership of the parent task
- Frontend uses only the public anon/publishable key — never the service role key

Security note: `.env` is gitignored. Vercel stores `VITE_SUPABASE_URL` and the public key as environment variables. The service role key is not used anywhere in this app.

Representative SQL (tasks + RLS)

```
create table public.tasks (  
  id uuid primary key default gen_random_uuid(),  
  user_id uuid not null references auth.users(id) on delete cascade,  
  title text not null,  
  description text,  
  status text not null default 'todo'  
  check (status in ('todo', 'in_progress', 'in_review', 'done')),  
  priority text not null default 'normal'  
  check (priority in ('low', 'normal', 'high')),  
  due_date date,  
  assignee_id uuid references public.team_members(id) on delete set null,  
  position integer not null default 0,  
  created_at timestamptz not null default now(),  
  updated_at timestamptz not null default now()  
);  
  
alter table public.tasks enable row level security;  
  
create policy "Users manage own tasks"  
  on public.tasks for all
```

```
using (auth.uid() = user_id)  
with check (auth.uid() = user_id);
```

Setup Instructions & Future Improvements

Run locally

```
git clone https://github.com/saileshpulukuri/kanban-board.git
cd kanban-board
npm install
cp .env.example .env
# Set VITE_SUPABASE_URL and VITE_SUPABASE_ANON_KEY
npm run dev
```

Supabase + deploy

1. Create a free Supabase project and run `supabase/schema.sql` in the SQL Editor.
2. Enable **Authentication** → **Providers** → **Anonymous**.
3. Copy Project URL + anon/publishable key into local `.env`.
4. In Vercel, set the same two environment variables and redeploy.
5. Open the live URL — a guest session is created automatically on first load.

Useful commands

- `npm run dev` — local development
- `npm run build` — production build
- `npm run preview` — preview build

Links

- [Live app](#)
- [GitHub repo](#)
- Schema: `supabase/schema.sql`

Tradeoffs & what I'd improve next

- **Realtime sync:** Supabase Realtime so multiple tabs stay in sync without refresh.
- **Stronger reorder writes:** batch column position updates in a single RPC/transaction.
- **Accessibility:** deeper keyboard reordering and screen-reader announcements for drag results.
- **Shared workspaces:** optional multi-user boards beyond per-guest isolation.
- **Onboarding:** richer empty-state guidance for brand-new guest accounts.

Evaluation summary

CRITERIA	EVIDENCE
Design quality	Intentional typography, teal/slate system, polished cards & drawer
Board functionality	4 columns, drag-and-drop, create/edit/delete
Persistence & security	Supabase schema + anonymous auth + RLS
Advanced features	Team, labels, comments, activity, filters, due urgency, stats
Live execution	Deployed at the Vercel URL above

Thank you for reviewing Flowboard. The live demo and repository contain everything needed to verify functionality, schema, and design quality end-to-end.